

Arduino Sketch Code Explanation

This folder contains two Arduino sketches: one for reading heart-rate data from a MAX30102/MAX30105 optical sensor, and one for measuring impedance with an AD5933 impedance analyzer.

1. MAX30102 Heart-Rate Sketch

File: `First_working_mx/First_working_mx.ino`

This sketch reads infrared data from a MAX30102/MAX30105 pulse sensor over I2C and estimates heart rate from peaks in the IR signal.

Libraries and Sensor Object

```
#include <Wire.h>

#include "MAX30105.h"

MAX30105 particleSensor;
```

`Wire.h` provides I2C communication. `MAX30105.h` is the sensor library used to control the MAX30102/MAX30105 module.

Important Variables

- `RATE_SIZE` is set to 4, so the code averages the last four valid BPM readings.
- `rates[]` stores recent BPM values.
- `lastPeakTime` stores the time of the previous detected heartbeat peak.
- `bpmAvg` stores the averaged heart-rate value.
- `irMin` and `irMax` track the minimum and maximum IR values in the current signal range.
- `risingEdge` helps detect when the waveform is moving upward toward a

peak.

- `IR_THRESHOLD` is set to 50000 and is used to decide whether a finger is present.

Setup Function

The setup starts serial communication at 115200 baud, waits two seconds, and starts I2C using ESP32 pins GPIO25 for SDA and GPIO26 for SCL:

```
Wire.begin(25, 26);
```

The code then checks whether the sensor is found. If not, it prints a status message and stops. If the sensor is found, it configures the sensor for red and IR LED mode with a 100 Hz sample rate.

Main Loop

Each loop reads the IR value:

```
long irValue = particleSensor.getIR();
```

If the IR value is below the threshold, the code assumes no finger is on the sensor. It resets the heartbeat detection variables and prints:

```
NO_FINGER, 0, 0, 0
```

If a finger is detected, the sketch tracks the current minimum and maximum IR values, calculates the signal range, then calculates the midpoint between the minimum and maximum.

Peak Detection

The code watches for the signal to cross the midpoint upward. When the signal reaches the tracked maximum after rising, it treats that as a heartbeat peak. The time difference between two peaks is converted to BPM:

```
float bpm = 60000.0 / delta;
```

The value 60000 is the number of milliseconds in one minute. For example, if two peaks are 1000 ms apart, the heart rate is 60 BPM.

The code only accepts peak intervals between 300 ms and 1500 ms, corresponding roughly to 200 BPM down to 40 BPM. This filters out unrealistic readings.

Serial Output

The output is formatted for a Python GUI or serial parser:

```
DATA, IR_VALUE, IR_RANGE, AVG_BPM
```

For example:

```
DATA, 74320, 1850, 72
```

2. AD5933 Impedance Measurement Sketch

File: `Impedance/Impedance.ino`

This sketch controls an AD5933 impedance analyzer over I2C. It performs a frequency sweep, calibrates the system with a known resistor, then measures an unknown RC circuit.

I2C and Register Definitions

The code uses ESP32 GPIO26 as SDA and GPIO27 as SCL:

```
#define SDA_PIN 26
```

```
#define SCL_PIN 27
```

The AD5933 I2C address is defined as:

```
#define AD5933_ADDR 0x0D
```

The register definitions such as `REG_CONTROL_HB`, `REG_START_FREQ`, `REG_REAL_DATA`, and `REG_IMAG_DATA` are addresses inside the AD5933 chip.

Sweep Settings

```
const double START_FREQ = 1000.0;

const double FREQ_STEP  = 1000.0;

const int NUM_POINTS = 100;
```

The sweep starts at 1 kHz, increases by 1 kHz at each point, and measures 100 points. This gives a sweep from 1 kHz to 100 kHz.

Low-Level I2C Helper Functions

- `writeRegister()` writes one byte to an AD5933 register.
- `readRegister()` reads one byte from an AD5933 register.
- `write24()` writes a 24-bit value across three registers.
- `write16()` writes a 16-bit value across two registers.
- `read16()` reads a signed 16-bit result from two registers.

Frequency Code Calculation

The AD5933 does not receive frequency directly in hertz. The sketch converts each frequency into the numeric tuning code required by the chip:

```
long frequencyCode(double frequency) {
```

```
return (long)((frequency * pow(2, 27)) / (MCLK / 4.0));  
  
}
```

Setting Up a Sweep

`setupSweep()` sends the start frequency, frequency increment, number of increments, and settling cycles to the AD5933. Then it initializes the start frequency and begins the sweep.

Calibration

The function `calibrateSystem()` is started by sending the character `c` over serial. During calibration, a known resistor should be connected between VOUT and VIN.

For each frequency point, the AD5933 returns a real and imaginary value. The code calculates magnitude and phase:

```
double magnitude = sqrt((double)realValue * realValue +  
  
                        (double)imagValue * imagValue);  
  
  
double phase = atan2((double)imagValue, (double)realValue);
```

The gain factor is calculated using the known calibration resistor:

```
gainFactor[i] = (1.0 / CALIBRATION_RESISTOR) / magnitude;
```

The measured system phase is also saved so it can be subtracted later during unknown measurements.

Unknown RC Measurement

The function `measureUnknownRC()` is started by sending the character `m` over serial. It performs the same frequency sweep, reads real and imaginary values, and calculates impedance magnitude:

```
double zMagnitude = 1.0 / (gainFactor[i] * magnitude);
```

Then it removes the calibration phase offset:

```
double impedancePhase = measuredPhase - systemPhase[i];
```

Finally, it separates impedance into resistance and reactance:

```
double resistance = zMagnitude * cos(impedancePhase);
```

```
double reactance = zMagnitude * sin(impedancePhase);
```

Serial Commands

- Send `c` to calibrate the system.
- Send `m` to measure the unknown RC circuit.

Serial Output

Calibration output:

```
Frequency, Real, Imag, Magnitude, RawPhaseDeg, GainFactor
```

Measurement output:

Frequency, Real, Imag, |Z| Ohm, Phase Deg, Resistance R Ohm, Reactance X Ohm

Important Note

The AD5933 sketch prints a message telling the user to connect a known 10k resistor, but the code defines the calibration resistor as 16k ohm:

```
const double CALIBRATION_RESISTOR = 16000.0;
```

These should match. Either the printed message should say 16k, or the constant should be changed to 10000.0 if a 10k resistor is actually used.